

Impression OS

Professionele WordPress-sites genereren met Elementor Pro

Wat is dit?

Een simpele maar complete gids: wat Impression OS is, hoe de architectuur in elkaar zit, en hoe je van een klant-briefing naar een volledige, professionele website komt — deterministisch en toegankelijk.

1 · Introductie	Wat het is (en niet is)
2 · Architectuur	De lagen: tokens → foundation → components → recipes → prompts → builder
3 · Aan de slag	Vereisten en snelstart
4 · De CLI	Alle commando's
5 · Van brief tot site	Enkele en meerdere pagina's
6 · Thema's	Merk-thema's met gegarandeerd contrast
7 · Uitbreiden	Token, component, recipe, thema toevoegen
8 · Voorbeelden & kwaliteit	De meegeleverde sites en de test-harness

1 · Introductie

Impression OS is een AI-native besturingssysteem voor het genereren van professionele WordPress-websites met Elementor Pro.

Het is geen website en geen Elementor-template. Het is het **systeem dat die dingen maakt**: een gestructureerde, machine-leesbare ontwerptaal plus een generator, zodat een AI-agent (of een eenvoudig commando) complete sites samenstelt zoals een ervaren ontwerp-team dat zou doen — van tokens, naar componenten, naar secties, naar een importeerbare Elementor Pro-kit.

Het probleem dat het oplost

- **Niet-determinisme** — losse AI-tools verzinnen ontwerpen ter plekke; twee runs geven twee resultaten.
- **Incoherentie** — één sectie kan er goed uitzien, maar een hele site drijft weg zonder gedeelde bron.
- **Ontoegankelijkheid** — contrast- en semantiek-fouten zijn de norm.

Impression OS haalt het improviseren weg: elke ontwerpbeslissing leeft als data. De generator **componeert** bekende, geteste bouwstenen in plaats van nieuwe te verzinnen.

Kernprincipes

- Ontwerpbeslissingen zijn data — herbruikbare waarden zijn tokens, nooit letterlijke waarden.
- Composeer, improviseer niet — de generator kiest primitives, verzint er geen.
- Deterministisch — dezelfde briefing + versie geeft dezelfde site (byte-stabiel).
- Toegankelijkheid is een contract — contrast, semantiek en focus worden afgedwongen, niet 'gereviewd'.
- De kit is het resultaat — schone, bewerkbare Elementor-output. Geen lock-in.

2 · Architectuur

Impression OS is gelaagd. Elke laag wordt één keer geschreven, geversioneerd, en gebruikt door de laag erboven. Een laag mag alleen naar **beneden** verwijzen — nooit naar boven.

prompts	De intentie-laag – hoe een briefing een build-plan wordt
recipes	Samengestelde secties (hero, pricing, testimonial, footer)
components	Atomaire units (button, card, heading, media, stat)
foundation	De ontwerp-wetten (grid, ritme, hierarchie, ally-contracten)
tokens	Ontwerpbeslissingen als data (kleur, type, ruimte, radius)
de Builder compileert dit alles tot een Elementor Pro-kit	

De **Builder** is de compiler die deze lagen doorloopt en een Elementor Pro-kit uitstoot: globale stijlen plus importeerbare pagina-/sectie-templates.

tokens/	De enige bron van waarheid voor elke ontwerpwaarde (kleur, type, ruimte, radius, schaduw).
foundation/	Ontwerp-wetten: grid, ritme, typografie, en de WCAG 2.1 AA-contracten.
components/	Atomaire bouwstenen, gespecificeerd als data en gekoppeld aan Elementor-widgets.
recipes/	Complete secties, samengesteld uit componenten, met een content-contract.
prompts/	Zet een briefing om in een deterministisch build-plan; guardrails.
builder/	De compiler: lagen → Elementor Pro-kit (globals + templates).
tools/	De impression-CLI en hulpmiddelen.
tests/	De harness: schema, referenties, contrast, determinisme, regressie.

3 · Aan de slag

Vereisten

- **Node.js 20 of hoger.** Verder niets — de builder, CLI en harness gebruiken alleen de Node-standaardbibliotheek (**nul dependencies**).
- De repository: github.com/impressdev/impression-os.

Snelstart

Draai alles vanuit de hoofdmap van de repo:

```
# 1. Bekijk wat het systeem biedt
node tools/bin/impression.js list recipes

# 2. Controleer alle databestanden (schema's + referenties)
node tools/bin/impression.js validate

# 3. Genereer een kit uit een voorbeeld-plan
node tools/bin/impression.js build examples/northwind/plan.json --out dist

# 4. Draai de kwaliteits-harness
node --test
```

Is alles groen, dan respecteert je omgeving elk contract dat het systeem afdwingt.

4 · De CLI

Eén commando, nul dependencies. Alle deelcommando's:

<code>impression validate</code>	Controleert elk databestand tegen zijn schema en referentie-integriteit.
<code>impression list</code>	Toont recipes, components of themes.
<code>impression new</code>	Scaffoldt een minimale, schema-valide briefing.
<code>impression plan</code>	Vouwt een briefing uit tot een enkel-pagina build-plan (zonder LLM).
<code>impression plan-site</code>	Vouwt een briefing uit tot een meerpagina site-plan (zonder LLM).
<code>impression build</code>	Compileert een build-plan tot een Elementor-kit + templates.
<code>impression build-site</code>	Compileert een site-plan tot één kit + per-pagina templates.
<code>impression lint</code>	Draait de guardrails tegen een build-plan.
<code>impression resolve-theme</code>	Bepaalt welk thema een briefing oplevert.
<code>impression theme ...</code>	Genereert een merk-thema (contrast-gegarandeerd AA).

Elk commando neemt optioneel `--root <repo>` (standaard de huidige map) en waar van toepassing `--out <pad>`.

5 · Van brief tot site

De hele keten draait deterministisch in code. De optionele LLM-planner is er voor briefings die oordeel vragen; voor standaardgevallen volstaat het code-pad volledig.

```
briefing ■■ (site-)plan ■■ Elementor Pro-kit ■■ WordPress
```

Enkele pagina

```
# Maak een lege briefing en vul 'm
node tools/bin/impression.js new Acme --out acme.brief.json

# Briefing -> plan -> kit
node tools/bin/impression.js plan acme.brief.json --out acme.plan.json
node tools/bin/impression.js lint acme.plan.json
node tools/bin/impression.js build acme.plan.json --out dist
```

Output: `dist/kit.json` (globale stijlen), `dist/templates/<sectie>.json` (per sectie) en `dist/page.json` (SEO/Open Graph, afgeleid uit de briefing).

Meerdere pagina's

Zet meerdere pagina's in de briefing (pages: `[{type, path}]`). Ze delen één thema en één kit; elke pagina krijgt zijn eigen blueprint.

```
node tools/bin/impression.js plan-site acme.brief.json --out site.json
node tools/bin/impression.js build-site site.json --out dist
# dist/kit.json, dist/site.json, dist/pages/<slug>/{templates/*, page.json}
```

Importeren in WordPress

- Pas `kit.json` toe via **Elementor** → **Site Settings** (globale kleuren, fonts, layout).
- Importeer elke sectie-template via **Templates** → **Saved Templates** en zet de pagina in volgorde samen.
- Elke sectie verwijst al naar de globals, dus de styling is meteen consistent.

6 · Thema's

Een thema levert de semantische **kleur**-rollen. **light** is de basis; **dark** en merk-thema's zijn delta's. De veiligste manier om een merk-thema toe te voegen is het te laten **genereren** — de generator kiest de accent-stappen op basis van contrast, zodat het resultaat gegarandeerd WCAG 2.1 AA haalt.

```
# thema op een bestaande ramp
node tools/bin/impression.js theme acme --accent violet --base light

# of: synthetiseer een ramp uit een enkele merk-kleur
node tools/bin/impression.js theme acme --hex "#7c3aed"
```

Kan geen enkele stap het contract halen, dan **faalt** de generator — liever een foutmelding dan een ontoegankelijk thema. De toegankelijkheids-harness controleert vervolgens elk thema in het manifest.

Voorbeeld uit de praktijk: het handgeschreven merk-thema kwam ooit op 4,47:1 uit — net onder AA. De harness ving het; regenereren bracht het op 5,93:1. Handwerk drift, generatie is gegarandeerd.

7 · Uitbreiden

Elke uitbreiding volgt hetzelfde patroon: schrijf de spec als data, koppel aan tokens, en draai de harness. Uitgebreide gidsen staan in docs/guides/.

Een token	Voeg de ruwe waarde toe in tokens/primitives/ en (indien nodig) een semantische rol.
Een component	Schrijf components//.json volgens het component-schema (props, states, a11y, Elementor).
Een recipe	Schrijf recipes/sections/.json (layout uit primitives, content-contract) en zet 'm in een blueprint.
Een thema	Gebruik 'impression theme' — contrast-gegarandeerd — of schrijf een delta met de hand.

De gouden regels: **componeer uit het systeem** (geen ruwe waarden boven de token-laag), **verwijs alleen naar beneden**, en **lever de test en de docs mee** met de functionaliteit.

8 · Voorbeelden & kwaliteit

Meegeleverde voorbeelden

Elk voorbeeld is byte-stabiel gereproduceerd uit zijn briefing en wordt door CI geverifieerd.

northwind	Een volledige landingspagina (8 secties) op het light-thema.
lumen	Een lead-pagina met contactformulier op het brand.teal-thema.
atlas	Een meerpagina site (landing + about) op het dark-thema — één gedeelde kit.

De kwaliteits-harness

Eén commando, `node - - test`, dwingt zes garanties af en gate't elke wijziging via CI:

- **Schema** — elke token, component, recipe, briefing en plan is structureel geldig.
- **Referenties** — geen losse token-verwijzingen; componenten mijden primitieve kleuren/fonts.
- **Toegankelijkheid** — tekst $\geq 4,5:1$, non-text $\geq 3:1$, in elk thema.
- **Determinisme** — dezelfde invoer geeft byte-stabiele uitvoer.
- **Guardrails** — build-plannen blijven binnen het systeem.
- **Regressie** — de output blijft gelijk aan goedgekeurde baselines.

Meer lezen in de repo: README.md (overzicht), PROJECT.md (visie & scope), ROADMAP.md (voortgang), CONTRIBUTING.md en docs/ (ADR's & gidsen).

Impression OS · Versie 0.1.0 · MIT · Gegeneerd als onderdeel van de projectdocumentatie.